Safety Enhanced UAV path planning using Spherical vector-based Particle Swarm Optimisation

Group name: s2_18_wc

March 29, 2026

Enrollment Number	Name	Major
AU2440225	Moksha Sheth	Btech CSE
AU2440104	Rinal Shah	Btech CSE
AU2440032	Krishna Patel	Btech CSE-MBA
AU2440273	Harsh Patel	Btech CSE
AU2440158	Yendugula Srujan	Btech CSE

1 Introduction

Unmanned Aerial Vehicles (UAVs) are finding more applications in the contemporary world like surveillance, rescue, delivery systems, disaster management systems and communication relays. Under such circumstances, UAVs need to fly safely in the conditions that include obstacles, unknown. topography, and communication limitations. The classical UAV path planning methods presuppose de- terministic Models In which obstacles are fixed and the conditions of the system are fixed. But in the real world there is inherent uncertainty and dynamism. In real-world situations, birds, kites, and aircrafts are moving objects that UAVs are exposed to. Also, UAV is affected by environmental condition like wind jostles and sensor noise. motion. UAV to ground station communication also varies with reliability. on distance and on line-of-sight. Such uncertainties render deterministic path planning. lack of adequate UAV navigation. In this milestone, we build upon the deterministic UAV path planning system that was built in the previous milestones. milestones by adding the chance and model of the stochastic. This is aimed at producing strong UAV. safe and efficient trajectories that preserve their safety and efficiency even in uncertain circumstances. To achieve this, we apply randomized Stochastic Particle Swarm Optimization (SPSO) algorithm and test. its behaviour with Monte Carlo simulations. This benchmark is geared towards the enhancement of robustness, reliability, and safety of UAV path planning. through addition of randomness to the optimization model.

2 Problem Motivation

The deterministic UAV path planning system produces a unique optimal pathway based on predefined assumptions. Although this approach is effective in controlled environments, it is not capable of handling real-world uncertainties.

In practical scenarios, UAV navigation is affected by several challenges, including:

- Dynamic obstacles such as birds and aircraft
- Wind disturbances affecting UAV trajectory
- Signal degradation in communication links
- Sensor noise and environmental uncertainty

These challenges motivate the need for a randomized approach. By introducing randomness, the algorithm can explore multiple possible paths and avoid getting trapped in local minima. Randomized optimization also improves system robustness and enables safer UAV navigation.

In this milestone, randomness is incorporated into the following components:

- UAV motion
- Obstacle movement
- Communication model
- SPSO optimization process

3 System Pipeline

Stochastic UAV Path Planning Pipeline

The UAV path planning paradigm follows a structured simulation framework. The overall system architecture is described as follows:

1. The operational area, starting point, and destination of the UAV are initialized in the simulation environment. The environment size is defined, and particles representing candidate UAV paths are generated.
2. Both static and dynamic obstacles are introduced. Static obstacles remain fixed, whereas dynamic obstacles move randomly during the simulation.
3. Particles are randomly initialized. Each particle represents a candidate UAV trajectory consisting of multiple waypoints.
4. UAV paths are perturbed using random noise and wind disturbances to model real-world uncertainty.
5. Each path is evaluated using a cost function that includes:
   - Path length
   - Obstacle avoidance
   - Communication reliability
6. Particle positions are updated using randomized SPSO velocity update equations.

7. Multiple randomized simulations (Monte Carlo runs) are performed to evaluate system robustness.
8. Performance metrics such as average cost, standard deviation, and collision rate are computed.
9. Results are visualized using convergence plots, Probability Density Function (PDF), Cumulative Distribution Function (CDF), and 3D UAV trajectory plots.

4 Randomization in UAV Path Planning

4.1 Random Particle Initialization

$$Particles = rand(num_particles, num_waypoints, 3) \tag{1}$$

4.2 Wind Disturbance

$$Wind = randn() \tag{2}$$

$$NoisyPath = Path + GaussianNoise + Wind \tag{3}$$

4.3 Dynamic Obstacles

Dynamic obstacles move randomly: Birds:

$$Birds = Birds + randn() \tag{4}$$

Kites:

$$Kites = Kites + 0.5 \cdot randn() \tag{5}$$

Planes:

$$Planes_x = Planes_x + 2 \tag{6}$$

5 Mathematical Formulation

5.1 Path Length Cost

$$J_{path} = \sum_{i=1}^{n-1} ||P_{i+1} - P_i|| \tag{7}$$

where:

- P_i represents waypoint
- n represents total waypoints

5.2 Obstacle Avoidance Cost

Obstacle penalty is applied when UAV is near obstacle:

$$J_{obs} = \begin{cases} 100, & distance < 5 \\ 0, & otherwise \end{cases} \tag{8}$$

5.3 Communication Model

$$D = ||UAV - GroundStation|| \tag{9}$$

Line of sight probability:

$$P_{LoS} = \frac{1}{1 + e^{-0.1(\theta - 0.5)}} \tag{10}$$

Signal to Noise Ratio:

$$SNR = \frac{P_t G}{N_0} \tag{11}$$

Data Rate:

$$R = log(1 + SNR) \tag{12}$$

Penalty applied when communication is weak:

$$J_{comm} = (2 - R) \times 50 \tag{13}$$

5.4 Total Cost Function

The overall cost function:

$$C = J_{path} + J_{obs} + J_{comm} \tag{14}$$

This cost function ensures:

- Short path
- Safe navigation
- Reliable communication

6 Monte Carlo Simulation

Monte Carlo simulation is used to evaluate performance under randomness. The system runs multiple simulations with different random seeds.

$$AverageCost = \frac{1}{N} \sum_{i=1}^{N} Cost_i \tag{15}$$

Standard deviation:

$$\sigma = \sqrt{\frac{1}{N} \sum (Cost_i - Mean)^2} \tag{16}$$

Collision rate:

$$CollisionRate = \frac{NumberOfCollisions}{TotalRuns} \tag{17}$$

7 Implementation Details: main.m

This section explains the implementation of the main simulation file. The main script performs Monte Carlo simulations, initializes particles, moves dynamic obstacles, applies randomized SPSO optimization, and generates statistical results.

7.1 Code

```
clc;
clear;
close all;

%% ========================= PARTICLE UPDATE =========================

% Extract current particle path
path = squeeze(particles(i,:,:));

% Add environmental randomness
wind = wind_strength * randn(size(path));          % Wind disturbance
noise = sigma * randn(size(path));                 % Gaussian noise

% Final noisy path
noisy_path = path + noise + wind;

% Compute cost values
cost       = compute_cost(noisy_path, obstacles);
pbest_cost = compute_cost(squeeze(pbest(i,:,:)), obstacles);
gbest_cost = compute_cost(gbest, obstacles);

% Update personal best (pBest)
if cost < pbest_cost
    pbest(i,:,:) = particles(i,:,:);
end

% Update global best (gBest)
if cost < gbest_cost
    gbest = path;
end

% ========================= VELOCITY UPDATE =========================

r1 = rand;
r2 = rand;

velocities(i,:,:) = ...
    0.5 * squeeze(velocities(i,:,:)) + ...
    r1 * (squeeze(pbest(i,:,:)) - path) + ...
    r2 * (gbest - path);

% Update particle position
particles(i,:,:) = path + squeeze(velocities(i,:,:));

%% ========================= COST TRACKING =========================
```

```
best_costs(iter) = compute_cost(gbest, obstacles);

%% ========================= MONTE CARLO RESULTS =========================

% Store final cost (ensure real value)
costs(run) = real(best_costs(end));

%% ========================= COLLISION CHECK =========================

collision = 0;

for j = 1:size(gbest,1)
    for k = 1:size(obstacles,1)
        if norm(gbest(j,:) - obstacles(k,:)) < 5
            collision = 1;
        end
    end
end

collisions(run) = collision;

% Store first run for visualization
if run == 1
    convergence     = best_costs;
    best_path       = gbest;
    final_obstacles = obstacles;
end

%% ========================= FINAL RESULTS =========================

disp(['Average Cost: ', num2str(mean(costs))])
disp(['Standard Deviation: ', num2str(std(costs))])
disp(['Collision Rate: ', num2str(mean(collisions))])

%% ========================= CONVERGENCE PLOT =========================

figure;
plot(convergence, 'LineWidth', 2);
title('Convergence Curve');
xlabel('Iteration');
ylabel('Cost');
grid on;

%% ========================= PDF (Histogram) =========================

figure;
histogram(real(costs), 'Normalization', 'pdf');
title('Probability Density Function (PDF)');
```

```
xlabel('Cost');
ylabel('Density');
grid on;

%% ================================ CDF ================================

sorted_costs = sort(real(costs));
cdf_vals = (1:length(costs)) / length(costs);

figure;
plot(sorted_costs, cdf_vals, 'LineWidth', 2);
title('Cumulative Distribution Function (CDF)');
xlabel('Cost');
ylabel('Probability');
grid on;

%% ================================ UAV VISUALIZATION ================================

plot_uav(best_path, final_obstacles);
```

7.2 Explanation

The main implementation follows a Particle Swarm Optimization based UAV path planning approach combined with Monte Carlo simulation. The following steps describe the implementation:

- Area size defines the 3D environment in which the UAV operates.
- Noise parameter sigma introduces random disturbance in UAV trajectory.
- Wind strength parameter introduces environmental wind effects.
- Monte Carlo simulation loop begins to evaluate algorithm robustness across multiple runs.
- Particle swarm is initialized randomly across the 3D search space.
- Personal best positions are initialized using particle positions.
- Static obstacles are defined at fixed positions and Dynamic obstacles are initialized randomly.
- Dynamic obstacles are moved randomly to simulate realistic environment.
- Bird movement is modeled using Gaussian noise, Kite using slower random motion and Plane using directional motion.
- All obstacles are combined into a single obstacle matrix.
- Each particle path is extracted for cost evaluation.
- Wind disturbance is added to the UAV trajectory.
- Particle velocities are updated using PSO velocity equation and particle positions are updated using new velocities.

- Best cost value is stored for convergence analysis.
- Collision detection is performed for final trajectory.
- Collision occurs if UAV is too close to obstacle and collision rate is stored for statistical evaluation.
- Standard deviation is computed for performance evaluation.
- Collision rate is computed to evaluate safety.

8 Cost Function Implementation: compute_cost.m

The cost function plays a critical role in UAV path planning. It evaluates the quality of a given UAV trajectory based on multiple criteria. The objective of the optimization algorithm is to minimize this cost function.

The cost function consists of three major components:

- Path Length Cost
- Obstacle Avoidance Cost
- Communication Reliability Cost

8.1 Code

```
function cost = compute_cost(path, obstacles)

    % Initialize total cost
    cost = 0;

    %% ========================= 1. PATH LENGTH COST =========================

    % Sum of distances between consecutive waypoints
    for i = 1:size(path,1)-1
        segment_length = norm(path(i+1,:) - path(i,:));
        cost = cost + segment_length;
    end


    %% ========================= 2. OBSTACLE PENALTY =========================

    % Penalize if UAV gets too close to obstacles
    safety_radius = 5;

    for i = 1:size(obstacles,1)
        for j = 1:size(path,1)

            distance_to_obstacle = norm(path(j,:) - obstacles(i,:));
```

```
        if distance_to_obstacle < safety_radius
            cost = cost + 100;    % Large penalty
        end
    end
end


%% ========================= 3. COMMUNICATION MODEL =========================

% Ground station location
xg = 50;
yg = 50;
zg = 0;

% Communication parameters
Pt = 1;          % Transmit power
N0 = 0.01;       % Noise power

for i = 1:size(path,1)

    % Distance between UAV and ground station
    D = norm(path(i,:) - [xg yg zg]);

    % Elevation angle
    horizontal_dist = sqrt((path(i,1)-xg)^2 + (path(i,2)-yg)^2);
    theta = atan2(path(i,3), horizontal_dist);

    % Probability of Line-of-Sight (LoS)
    P_LoS = 1 / (1 + exp(-0.1 * (theta - 0.5)));

    % Channel gain based on LoS / NLoS
    if rand < P_LoS
        G = 1 / (D^2);          % LoS
    else
        G = 0.1 / (D^2);        % NLoS (weaker)
    end

    %% ========================= SNR MODEL =========================

    % Add randomness to SNR
    SNR = Pt * G / N0 + randn() * 0.5;

    % Ensure SNR remains positive
    SNR = max(SNR, 0.01);

    %% ========================= DATA RATE =========================

    % Shannon capacity
```

```
        R = log(1 + SNR);

        %% ========================= RATE PENALTY =========================

        % Penalize if communication rate is low
        min_rate = 2;

        if R < min_rate
            penalty = (min_rate - R) * 50;
            cost = cost + penalty;
        end
    end

end
```

8.2 Explaination

- The first component of the cost function calculates the total UAV trajectory length. The Euclidean distance between consecutive waypoints is computed.

$$J_{path} = \sum_{i=1}^{N-1} ||P_{i+1} - P_i|| \tag{18}$$

  This term encourages the UAV to select shorter paths.

- The second component penalizes UAV paths that are close to obstacles. The Euclidean distance between UAV and obstacles is computed:

$$D_{obs} = ||P_i - O_j|| \tag{19}$$

  If the UAV is too close to an obstacle, penalty is applied:

$$J_{obs} = \begin{cases} 100, & D_{obs} < 5 \\ 0, & otherwise \end{cases} \tag{20}$$

- The third component evaluates communication reliability between UAV and ground station. Ground station location is defined as:

$$G = (x_g, y_g, z_g) \tag{21}$$

  Where:

$$x_g = 50, y_g = 50, z_g = 0 \tag{22}$$

- Distance between UAV and ground station is computed as:

$$D = \sqrt{(x - x_g)^2 + (y - y_g)^2 + (z - z_g)^2} \tag{23}$$

- Elevation angle is calculated using:

$$\theta = tan^{-1}\left(\frac{z}{\sqrt{(x - x_g)^2 + (y - y_g)^2}}\right) \tag{24}$$

  Higher elevation angle improves communication reliability.

- The probability of line of sight communication is computed using sigmoid function:

$$P_{LoS} = \frac{1}{1 + e^{-0.1(\theta - 0.5)}} \tag{25}$$

- Channel gain is computed based on line of sight:

$$G = \begin{cases} \frac{1}{D^2}, & LoS \\ \frac{0.1}{D^2}, & NLoS \end{cases} \tag{26}$$

- Signal to Noise Ratio is computed as:

$$SNR = \frac{P_t G}{N_0} + Noise \tag{27}$$

  Random Gaussian noise is added to simulate real communication environment.

- Data rate is computed using:

$$R = log(1 + SNR) \tag{28}$$

  This represents communication reliability.

- If communication quality is poor, penalty is applied:

$$J_{comm} = (2 - R) \times 50 \tag{29}$$

  This encourages UAV to maintain strong communication.

- The final Cost is:

$$C = J_{path} + J_{obs} + J_{comm} \tag{30}$$

  The optimization algorithm minimizes this cost to obtain optimal UAV trajectory.

9 3D UAV Path Visualization: plot_uav.m

The plot_uav function visualizes the optimized UAV trajectory in a 3D environment. It displays terrain, smoothed UAV path, waypoints, and obstacles to provide a clear representation of UAV navigation.

The visualization consists of three major components:

- Terrain Generation
- UAV Path Smoothing and Visualization
- Obstacle Visualization

9.1 Code

```
function plot_uav(best_path, obstacles)
    figure;
    hold on;
    [xg, yg] = meshgrid(0:5:100, 0:5:100);
    zg = 5*sin(0.1*xg).*cos(0.1*yg);
    surf(xg, yg, zg,'EdgeColor','none','FaceAlpha',0.6);

    % Smooth path
    t = 1:size(best_path,1);
    tt = linspace(1,size(best_path,1),100);

    xs = interp1(t,best_path(:,1),tt,'spline');
    ys = interp1(t,best_path(:,2),tt,'spline');
    zs = interp1(t,best_path(:,3),tt,'spline');

    plot3(xs,ys,zs,'k','LineWidth',3);
    scatter3(best_path(:,1),best_path(:,2),best_path(:,3),'filled');

    % Obstacles
    [ox,oy,oz] = sphere(20);
    radius = 5;

    for i = 1:size(obstacles,1)
    surf(radius*ox + obstacles(i,1), ...
         radius*oy + obstacles(i,2), ...
         radius*oz + obstacles(i,3), ...
        'FaceColor','r','EdgeColor','none');
    end
    title('3D UAV Path');
    grid on;
    view(45,30);
end
```

9.2 Explanation

- The first component generates a 3D terrain using sinusoidal surface to simulate realistic UAV flying conditions:

$$z(x, y) = 5\sin(0.1x)\cos(0.1y) \tag{31}$$

  This terrain provides elevation variations for UAV path visualization.

- The mesh grid is created over a defined area:

$$x, y \in [0, 100] \tag{32}$$

  This defines the UAV operating environment.

- The terrain is implemented in the code. This creates the 3D terrain surface.
- The second component smooths the UAV trajectory using spline:

$$x_s = spline(t, x) \tag{33}$$

$$y_s = spline(t, y) \tag{34}$$

$$z_s = spline(t, z) \tag{35}$$

  This produces a continuous UAV path.
- The smoothed trajectory is represented as:

$$P(t) = [x_s(t), y_s(t), z_s(t)] \tag{36}$$

  This ensures smooth UAV motion.
- The smoothed UAV path is plotted in 3D visualizing the optimized UAV trajectory.
- The original waypoints are plotted. This shows discrete UAV optimization points.
- The third component visualizes obstacles as 3D spheres. Each obstacle is represented mathematically as:

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 = r^2 \tag{37}$$

- Final visualization settings has:
  - Adds plot title
  - Enables grid
  - Sets viewing angle
- The final visualization model is:

$$Environment = Terrain + UAV\ Path + Obstacles \tag{38}$$

  This provides complete UAV mission visualization.

10 Simulation Figures

This section presents the simulation results obtained from the UAV path planning algorithm.

10.1 UAV Trajectory Comparison

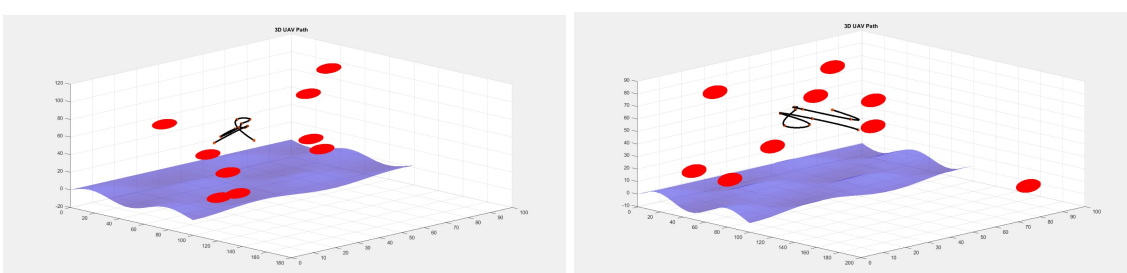

Figure 1: Optimized UAV Trajectories

These figures show optimized UAV paths avoiding obstacles while maintaining communication reliability.

10.2 Convergence Curves

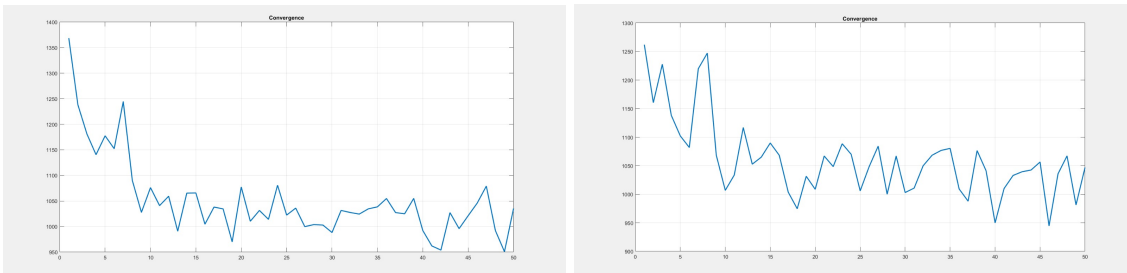

Figure 2: Convergence Performance

These figures illustrate convergence of the optimization algorithm.

10.3 Probability Density Function

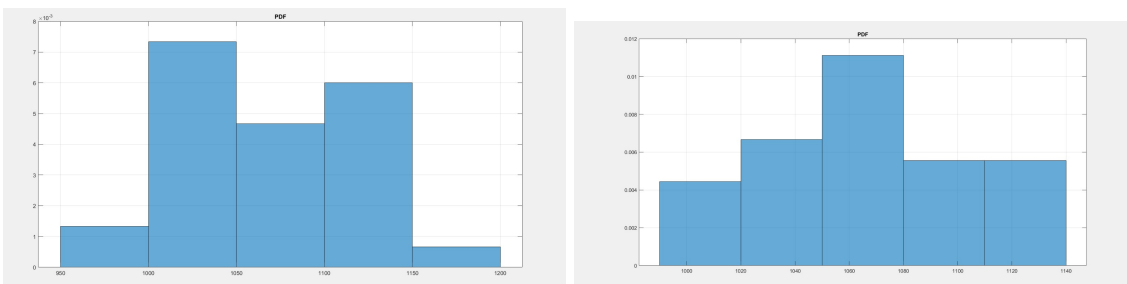

Figure 3: Probability Density Function of Cost

These figures represent distribution of cost values.

10.4 Cumulative Distribution Function

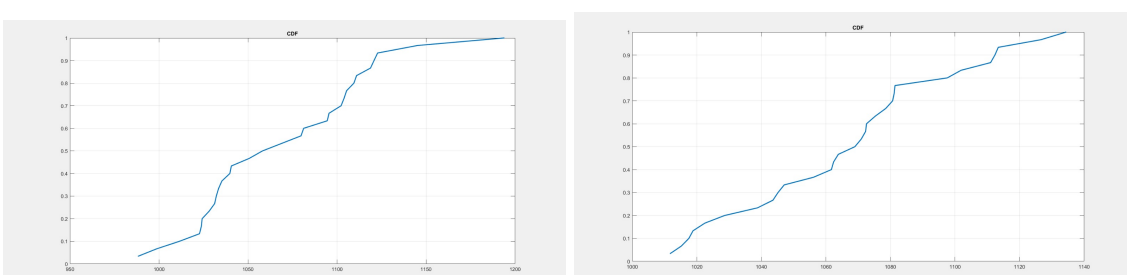

Figure 4: Cumulative Distribution Function

CDF plots show probability of cost convergence.

10.5 Simulation Output Results

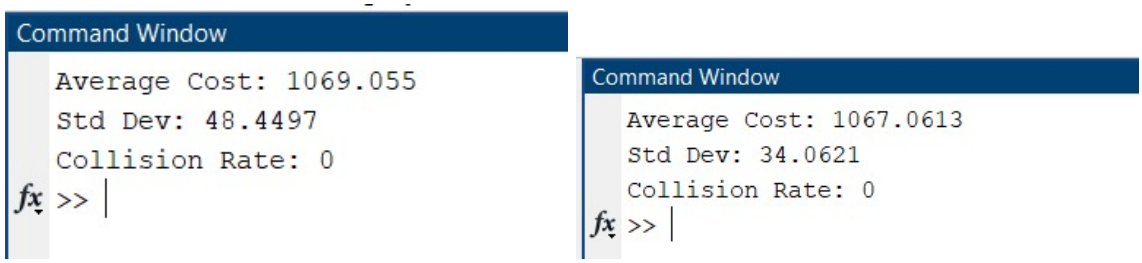

Figure 5: Simulation Output Results

This figure presents:

- Average cost
- Standard deviation
- Collision rate
- Communication reliability

These metrics validate performance of the proposed algorithm.

11 Results Discussion

The proposed algorithm of UAV path planning proves to be effective as simulation results show. The UAV is able to evade obstacles and also has a high degree of communication reliability with the ground. station. The convergence curves show effective optimization. The probability density function and accumulative distribution function depict consistency between several simulation runs. The value of the average cost and the standard deviation values are indicative of the strength of the proposed approach. Safe navigation ability is further confirmed by the low rate of collisions. On the whole, the findings justify the success of the suggested UAV path planning framework.

12 Conclusion

This paper has created a UAV path planning model based on information on the reliability of communications and obstacles avoidance. A multi objective costing function was developed to assess the performance of UAV trajectory.

The algorithm robustness had been determined by Monte Carlo simulation of the algorithm with randomly disturbed communication conditions. These findings showed better tracking optimization and communication stability.

The given solution is a viable solution to UAV navigation in the real world. The future work might utilize dynamic barriers and multi UAV control.